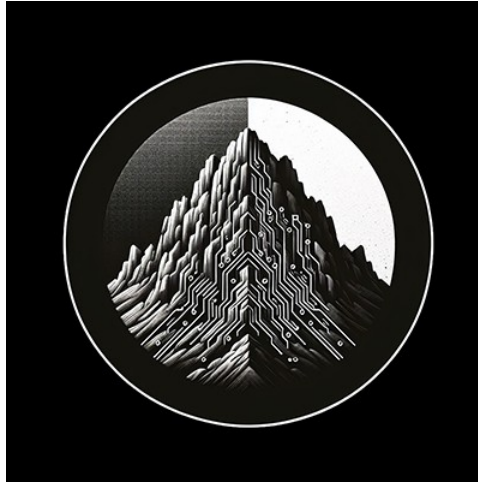




RTL Design Sherpa

DDR2/LPDDR2 Characterization Nexys A7 Project Guide 0.90

July 2026

Table of Contents

1 Document Information.....	8
1.1 References.....	8
1.2 Terminology.....	9
1.3 Revision History.....	9
2 Overview.....	9
2.1 What it characterizes.....	9
2.2 The three flows.....	10
2.3 Board and tools.....	10
2.4 Workload families.....	11
3 How It Works.....	11
3.1 The transport spine.....	11
3.1.1 Figure 2.1: DDR2 Characterization Harness Spine.....	12
3.2 The 1→4 AXIL bridge.....	12
3.3 Key RTL blocks.....	12
3.4 The characterization engines.....	13
3.5 The controller, DFI, and PHY (black boxes).....	13
4 Test Methodology.....	14
4.1 What is under test.....	14
4.2 What is measured, and how.....	14
4.3 Workloads.....	14
4.4 The oracle.....	15
4.5 Measurement pitfalls.....	15

5 Build and Run.....	15
5.1 flows-ours-uart make targets.....	15
5.2 The host programs (flows-ours-uart/host/).....	16
5.3 Typical bring-up sequence.....	17
6 Harness CSR Configuration.....	17
6.1 Bridge address map.....	17
6.2 By-name access (RDL → regmap).....	18
6.2.1 Waveform 5.1: Engine Kick and Poll.....	19
6.3 Control / status (offsets relative to HARNESS_CSR_BASE).....	19
6.4 Timer.....	20
6.5 Runtime controller cfg + DFI tuning.....	21
6.6 a7ddrphy calibration passthrough (hardware only).....	22
6.7 Engine config (WR 0x100–0x128, RD 0x180–0x1A8).....	22
6.8 Perf observability (0x1C0–0x1E8).....	23
7 Simulation / Silicon Equivalence.....	23
7.1 Testbenches.....	24
7.2 How the same program runs in sim.....	24
7.3 Match the board’s exact DFI config.....	24
7.4 What the sim can and cannot reproduce.....	25
8 Troubleshooting.....	25
8.1 Current board state.....	25
8.2 Sticky errors look like a wedge.....	25
8.3 Known bugs / bounds.....	25
8.4 Cannot find the board.....	26

8.5 a7ddrphy is not simulatable.....	26
8.6 debug_sram trace ring is disabled on this build.....	26
8.7 LiteDRAM baseline flow.....	26

List of Figures

Figure 2.1: DDR2 Characterization Harness Spine.....	12
--	----

List of Tables

Table 1: Reference documents.....	8
Table 2: Revision history.....	9
Table 3: The characterization flows.....	10
Table 4: Board and tool requirements.....	10
Table 5: Access-pattern families.....	11
Table 6: Key RTL blocks.....	12
Table 7: What is measured.....	14
Table 8: Characterization workloads.....	14
Table 9: flows-ours-uart make targets.....	15
Table 10: Bridge slave address map.....	17
Table 11: Control / status registers.....	19
Table 12: Characterization timer registers.....	20
Table 13: Runtime controller config and live DFI tuning.....	21
Table 14: a7ddrphy calibration passthrough.....	22
Table 15: Write-engine config (read engine identical at +0x80).....	22
Table 16: Perf observability (bin b covers $[2^b, 2^{(b+1)})$ cycles).....	23
Table 17: Simulation testbenches.....	24
Table 18: Known issues and bounds.....	25

List of Waveforms

Waveform 5.1: Engine Kick and Poll.....	19
---	----

1 Document Information

This is the operator / developer guide for the Nexys A7 **DDR2/LPDDR2 Characterization** project (projects/NexysA7/ddr2-characterization). It explains what the project characterizes, how the harness works, how to build / simulate / program / run it, and how the harness CSR is configured.

The memory controller under test — **pumice** — is separate IP (projects/components/memory-controllers/pumice-ddr2-lpddr2). This guide treats it as a **black box** driven by the characterization harness; it does not document the controller’s internals.

Note: the project’s top-level README.md still carries an early “skeleton — RTL not yet written” status line. That is stale: the harness, host, sim, and both flows are built and running. Trust this guide and the RTL/host sources over that status.

1.1 References

Reference documents

Source	Title
RTL Design Sherpa	flows-ours-uart/host/ ADDRESS_MAP.md (bridge + CSR map)
RTL Design Sherpa	flows-ours-uart/bin/ README_a7ddrphy.md (PHY regen)
RTL Design Sherpa	flows-litedram-uart/ HARNESS_PLAN.md (baseline flow)
RTL Design Sherpa	bin/TBClasses/harness/ (shared UART-char collateral)
Micron	MT47H64M16HR-25E DDR2 SDRAM datasheet
ARM	AMBA AXI4 / AXI4-Lite / APB Protocol Specifications

1.2 Terminology

DFI

DDR PHY Interface. The command/data protocol between the memory controller and the PHY. Here pumice emits a phase-packed flat DFI v2.1 bus.

a7ddrphy

LiteDRAM's Artix-7 DDR2 PHY (4:1 serdes, 800 Mbps). Uses Xilinx OSERDESE2/ISERDESE2/IDELAYE2 and therefore does **not** simulate in Verilator.

GEAR / DFI_RATE

The DFI phase count. $\text{GEAR_RATIO} = \log_2(\text{DFI_RATE})$. Must match between the RTL build and the sim model or reads corrupt.

Engine

A master-side AXI4 traffic generator: the write engine emits an LFSR data pattern; the read engine re-reads and CRC-checks it.

Leveling

Firmware-driven a7ddrphy read/write training over the PHY-CSR passthrough window; there is no hardware leveling FSM.

1.3 Revision History

Revision history

Version	Date	Notes
0.90	2026-07-18	Initial project guide

2 Overview

2.1 What it characterizes

This project performs end-to-end, board-level **performance, latency, and data-integrity characterization** of a DDR2/LPDDR2 memory controller on real silicon. It drives the onboard Micron MT47H64M16HR-25E DDR2 (x16, single-rank, 128 MiB, up to 800 Mbps) through the controller against the real device, and reports throughput (MB/s), latency histograms, and per-workload CRC integrity across burst-length / stride / gap / page-policy / scheme sweeps.

There is no soft CPU in the FPGA — an off-board Python **host** drives the harness over the FTDI USB-UART (115200 8N1). The same host programs run against a cocotb simulation (Chapter 6).

2.2 The three flows

The project is organized as sibling “flows” plus a shared framework:

The characterization flows

Flow	Controller (DUT)	What it is
flows-ours-uart/	pumice DDR2 (black box) + DFI + a7ddrphy	Primary flow: UART-driven host + Vivado build of the pumice harness on the Nexys A7.
flows-litedram-uart/	LiteDRAM litedram_core	Apples-to-apples baseline: LiteDRAM’s own DDR2 controller (self-initializing, own PLL/PHY) driven by the same engines/taps/host.
ddr2_char_framework/	—	Shared instrumentation: harness RTL, the engine wrapper (ddr2_char_macro), the 1→4 AXIL bridge, and the cocotb DV.

The naming convention is flows-<controller>-uart/ for on-Nexys builds (host over UART, no CPU). pumice is separate IP under projects/components/memory-controllers/pumice-ddr2-lpddr2/.

2.3 Board and tools

Board and tool requirements

Item	Detail
Board	Digilent Nexys A7-100T, Artix-7 xc7a100tcsg324-1

Item	Detail
DRAM	Onboard Micron MT47H64M16 DDR2 x16, ROW_WIDTH = 13
Build	Vivado (batch mode)
Host	Python 3 + pyserial
Sim	Verilator + cocotb / cocotb_test / pytest
PHY regen (only)	Python 3.10 + migen / litex 2024.12 / litedram 2024.12

2.4 Workload families

The characterization sweeps four access-pattern families (host/pumice_char.py):

Access-pattern families

Family	Access	Purpose
incremental	contiguous linear	best-case sequential
row_major	bounded to one page	guaranteed page HIT
col_major	stride = one row, same bank	guaranteed page MISS (worst case)
col_major_interleave d	stride = one bank	activates the bank pipeline

Controller-config presets (baseline, bank_interleave, open_page, inorder, reorder, happy_hybrid, fast_refresh, slow_refresh) each flip one lever from baseline; they are driven over the pumice APB slave (black box). Run profiles smoke / matrix / full are shared by sim and board.

3 How It Works

3.1 The transport spine

Host commands reach the DUT over one UART link that fans out to four AXI4-Lite slaves:

File	Role
	char timer + LED / 7-seg.
ddr2_char_framework/rtl/ ddr2_char_macro.sv	Binds the two AXI4 engines to pumice's s_axi, and holds the perf taps (bus meters + latency histogram) on the internal AXI wires.
ddr2_char_framework/rtl/ harness_csr.sv	The AXIL CSR slave (Chapter 5). Hand-written (self-clearing pulses, latches, PHY passthrough).
ddr2_char_framework/rtl/ dfi_v21_flat_to_a7ddrphy.sv	Combinational adapter: pumice's phase-packed flat DFI v2.1 → a7ddrphy per-phase ports (NPHASES = 4).
ddr2_char_framework/rtl/ a7ddrphy_stub.sv	Port-shape black box of a7ddrphy for Verilator (real body swapped in by Vivado).

3.4 The characterization engines

Inside ddr2_char_macro, two master-side AXI4 engines generate and check traffic against the controller's s_axi port:

- **Write engine** (axi4_master_wr_pattern_gen) — emits an LFSR data pattern across the programmed address pattern, and latches an expected CRC.
- **Read engine** (axi4_master_rd_crc_check) — re-reads the same pattern and computes an actual CRC, plus a per-beat mismatch count.

Alongside them, **perf taps** — axi_bus_meter (write and read) and axi_perf_latency_hist — observe the internal AXI wires to produce the producer/backpressure/starve/idle buckets and the latency histogram read back in Chapter 5.

3.5 The controller, DFI, and PHY (black boxes)

The controller (**pumice**) sits inside ddr2_char_macro between the AXI engines and the flat DFI output. Its configuration registers are reached over the ddr2_apb bridge slave at base 0x0. This guide documents only that the harness drives it — not its internals.

pumice's flat DFI v2.1 bus is adapted to the a7ddrphy per-phase interface, then to the DDR2 pads. **a7ddrphy does not simulate in Verilator** (it uses Xilinx SERDES/IDELAY primitives); the sim connects at the DFI level with a behavioral model instead (Chapter 6). On hardware, a7ddrphy

calibration is driven by firmware (the host) over the PHY-CSR passthrough window — there is no hardware leveling FSM.

4 Test Methodology

This chapter describes what is exercised on the FPGA block (Figure 2.1) and how throughput, latency, and data integrity are measured.

4.1 What is under test

The **pumice** memory controller (black box) driving the real onboard DDR2, stimulated by the two master-side AXI4 engines. Every measurement is taken on the internal AXI wires between the engines and the controller, so the numbers reflect the controller + PHY + device as a whole.

4.2 What is measured, and how

What is measured

Quantity	How it is obtained
Throughput	$\text{beats} \times \text{bytes_per_beat} / (\text{TIMER_CYCLES} \times 10 \text{ ns})$ — the timer bounds the run in sys_clk cycles
Latency histogram	OBS_HIST_SEL selects bus (rd/wr) + metric (AR→firstR / AR→RLAST) + bin; OBS_HIST_COUNT / OBS_HIST_TOTAL read it back. Bin b covers $[2^b, 2^{(b+1)})$ cycles
Data integrity	CRC_ACTUAL vs CRC_EXPECTED (CRC_MATCH), plus BEATS_MISM
Utilization	OBS_{RD,WR}_{PROD,BP,STARV,IDLE} bus-meter buckets

A measurement run: program the engine config (address pattern, burst length, txn count, LFSR seed), clear_stats, kick with CTRL.start_wr / start_rd, poll STATUS.wr_done / rd_done, then read the timer, CRC, histogram, and meters. The write engine latches the expected CRC as it generates the pattern; the read engine re-reads and computes the actual CRC.

4.3 Workloads

Characterization workloads

Axis	Values
Access pattern	incremental, row_major, col_major, col_major_interleaved

Axis	Values
Controller preset	baseline, bank_interleave, open_page, inorder, reorder, ... (over the pumice APB slave)
Burst / count / stride	swept per --char-profile (smoke / matrix / full)

4.4 The oracle

Integrity is judged against a deterministic **LFSR pattern + CRC-32**: the write engine's CRC_EXPECTED must equal the read engine's CRC_ACTUAL with BEATS_MISM = 0. The same golden logic runs in sim (Chapter 6).

4.5 Measurement pitfalls

- STATUS.any_error is **sticky** — clear it with clear_stats between phases or every later run looks wedged.
- Keep board runs $\text{txn} \leq 1024$ (read-engine CAM ceiling) for clean completion; the *rate* stays valid at scale.
- col_major stride $\times$ large txn can alias past the 128 MB device — bound it.
- The sim must be built at the board's exact DFI_RATE / GEAR or it passes while the board fails (Chapter 6).

5 Build and Run

All commands assume the Python environment is sourced (sets REPO_ROOT, SIM=verilator, PATH, PYTHONPATH):

```
cd /path/to/RTLDesignSherpa && source env_python
cd projects/NexysA7/ddr2-characterization/flows-ours-uart
```

5.1 flows-ours-uart make targets

UART ?= /dev/ttyUSB1, BAUD ?= 115200, VIVADO ?= vivado.

flows-ours-uart make targets

Target	Action
make lint	Verilator lint of ddr2_char_top via the harness filelist
make project / make synth	create Vivado project / synth-only +

Target	Action
	reports
<code>make bitstream</code>	full synth + impl + bitgen (~10–30 min) → <code>bitstream/ddr2_char.bit</code>
<code>make bitstream-ila</code>	same, plus an ILA on the DFI boundary → <code>.bit</code> + <code>.ltx</code>
<code>make program</code>	flash the board
<code>make utilization/make timing</code>	print the latest reports
<code>make smoke</code>	<code>host/run_smoke.py</code> — link + one linear WR/RD integrity pass
<code>make status</code>	<code>host/ddr2_char.py</code> — one-shot BUILD_ID + status dump
<code>make level</code>	<code>host/pumice_master.py --level-</code> <code>only</code> — a7ddrphy leveling
<code>make simple</code>	<code>host/pumice_master.py --simple</code> — init + one write→read pass
<code>make characterize</code>	<code>host/pumice_master.py --full</code> — full workload sweep
<code>make sweep-rddly/make train-</code> <code>deskew</code>	PHY timing sweeps
<code>make host-test</code>	<code>pytest host/test_pumice_master.py</code> (mock UART, no board)
<code>make sim/sim-smoke/sim-simple/</code> <code>sim-level</code>	run the same host programs in cocotb sim
<code>make clean/clean-all</code>	remove build artifacts

The other Makefiles: `flows-litedram-uart/Makefile` (`make regen/make bitstream/make program` for the LiteDRAM baseline) and `ddr2_char_framework/dv/tests/Makefile` (the cocotb macro suite: `make run`, `run-smoke`, `run-<shape>`, and `parallel run-{gate,func,full}-parallel`).

5.2 The host programs (`flows-ours-uart/host/`)

The port defaults to `--port auto`, which probes every `/dev/ttyUSB*` and keeps the board that answers `BUILD_ID == 0x44445232` (“DDR2”).

`run_smoke.py` — the first thing to run after flashing. Programs a linear WR+RD workload, kicks both engines, and checks CRC / cycles / perf.


```
python3 host/run_smoke.py --port /dev/ttyUSB1 --txn 1024 --blen 8 --seed 0xDEADBEEF
```

Pass = BUILD_ID matches, both engines done, CRC_ACTUAL == CRC_EXPECTED, BEATS_MISMATCHED == 0, and TIMER.pass.

pumice_master.py — the orchestration program; one mode is required:

```
python3 host/pumice_master.py --port /dev/ttyUSB1 --level-only # PHY read/write leveling
python3 host/pumice_master.py --port /dev/ttyUSB1 --simple # init + one write→read pass
python3 host/pumice_master.py --port /dev/ttyUSB1 --full # full workload sweep
python3 host/pumice_master.py --port /dev/ttyUSB1 --char --char-profile matrix --char-scale 1000 --csv out.csv
```

Useful options: --no-level, --rd-phase, --rd-delay, --char-configs, --char-scale N (≈1000 on FPGA), --level-cache JSON, --clk-mhz.

ddr2_char.py — the driver library plus a one-shot status CLI (python3 host/ddr2_char.py --port auto). It exports DDR2CharDriver, the base-address constants, and the enum constants used throughout.

5.3 Typical bring-up sequence

```
make bitstream
make program
make smoke UART=/dev/ttyUSB1 # link + integrity
make level # find the read/write eye
make simple # one clean write→read
make characterize # full sweep -> CSV
# no board? prove the same programs in sim:
make sim
```

6 Harness CSR Configuration

6.1 Bridge address map

The UART bridge fans out to four AXI4-Lite slaves. Bases come from bridge_ddr2_char_axil.toml (mirrored in host/ADDRESS_MAP.md and as constants in ddr2_char.py):

Bridge slave address map

Constant	Slave	Base	Size	Protocol
DDR2_APB_BASE	ddr2_apb	0x0000_00	4 KB	APB — pumice

Constant	Slave	Base	Size	Protocol
		00		controller CSR (black box)
HARNESS_CSR_BASE	harness_csr	0x0001_00	4 KB	AXIL — this chapter
DEBUG_SRAM_BASE	debug_sram	0x0004_00	256 KB	AXIL 64b — MonBus/DFI trace ring
DFI_MON_RAM_BASE	dfi_mon_ram	0x0008_00	4 KB	AXIL — DFI cmd observability

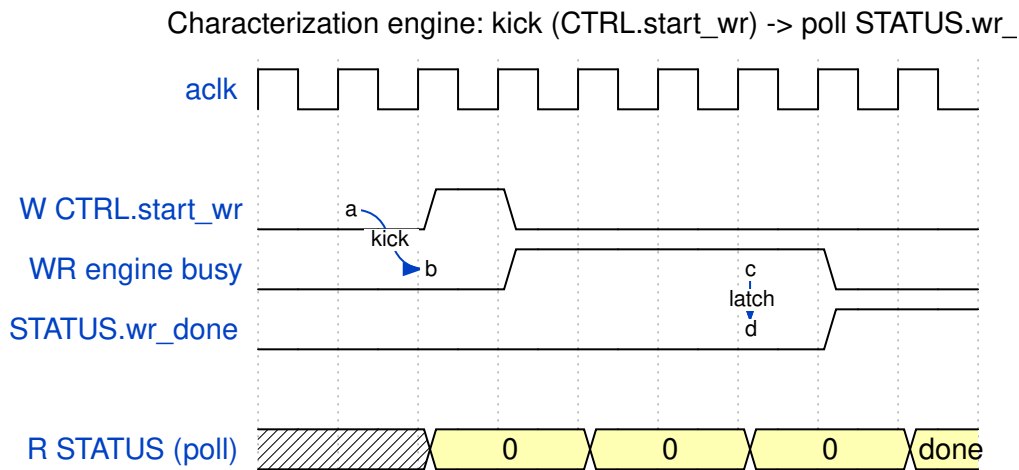
6.2 By-name access (RDL → regmap)

`harness_csr.sv` is hand-written; `harness_csr.rdl` is a descriptor of it, from which the by-name host regmap is generated. Registers are accessed by name via `UartRegisterMap`, never by hardcoded offset. Regenerate after editing either:

```
python3 bin/peakrdl_generate.py \
projects/NexysA7/ddr2-characterization/ddr2_char_framework/rtl/harness
_csr.rdl \
--regmap --docs-only --no-html --no-markdown \
--regmap-output
projects/NexysA7/ddr2-characterization/ddr2_char_framework/dv/tbclasses/
harness_csr_regmap.py
```

A consistency test (`host/test_harness_regmap_consistency.py`) asserts the generated regmap matches the SV header table.

6.2.1 Waveform 5.1: Engine Kick and Poll



Engine kick and poll

Source: [01_engine_kick.json](#)

6.3 Control / status (offsets relative to HARNESS_CSR_BASE)

Control / status registers

Register	Offset	Field	Bits	Access	Description
CTRL	0x00	start_w r	[0]	W	Pulse: kick the write engine
CTRL	0x00	start_r d	[1]	W	Pulse: kick the read engine
CTRL	0x00	clear_s tats	[2]	W	Pulse: clear perf + sticky errors
CTRL	0x00	freeze_ trace	[3]	RW	Latch: freeze perf counters
CTRL	0x00	soft_re set	[4]	W	Pulse: soft reset (does not clear sticky errors)
STATUS	0x04	wr_done / rd_done	[0] / [1]	R	Engine completion
STATUS	0x04	wr_erro r /	[2] / [3]	R	Per-engine error

Register	Offset	Field	Bits	Access	Description
STATUS	0x04	rd_error any_error	[4]	R	Sticky OR of errors — cleared by clear_stats
STATUS	0x04	init_done / init_fail	[6] / [7]	R	Controller init status
DBG_WR_PTR	0x08	value	[31:0]	R	Words written to debug_sram since clear
DBG_OVERFLOW	0x0C	overflow	[0]	R	Sticky trace-overflow
CRC_EXPECTED	0x10	value	[31:0]	R	Write-engine expected CRC
CRC_ACTUAL	0x14	value	[31:0]	R	Read-engine actual CRC
CRC_MATCH	0x18	match	[0]	R	CRC_ACTUAL == CRC_EXPECTED
CRC_MATCH	0x18	exp_val id / act_val id	[1] / [2]	R	CRC captured
CRC_MATCH	0x18	beats_mism_nz	[3]	R	Any beat mismatched
SCRATCH	0x1C	value	[31:0]	RW	Host link ping
BUILD_ID	0x20	value	[31:0]	R	0x44445232 (“DDR2”)
BEATS_MISM	0x24	value	[31:0]	R	Beats mismatched

6.4 Timer

Characterization timer registers

Register	Offset	Bits	Access	Description
TIMER_CTRL	0x28	[0] clear	W	Pulse: clear the cycle

Register	Offset	Bits	Access	Description
TIMER_STATUS	0x2C	[0] done, [1] running, [2] pass	R	counter Timer state
TIMER_CYCLES_LO/HI	0x30 / 0x34	[31:0]	R	64-bit cycle count (10 ns/cycle)
TIMER_EXP_BEATS	0x38	[31:0]	RW	Beat-count stop trigger (0 = disable)
TIMER_{R,W}_{FIRST, LAST}_{LO, HI}	0x40–0x5C	[31:0]	R	First/last R/W beat timestamps (64-bit each)

6.5 Runtime controller cfg + DFI tuning

Runtime controller config and live DFI tuning

Register	Offset	Field	Bits	Access	Description
CTRLR_CFG	0x60	memtype	[0]	RW	0 = DDR2, 1 = LPDDR2
CTRLR_CFG	0x60	t_phy_wrlat	[15:8]	RW	PHY write latency (see Chapter 7 — board wants 0)
CTRLR_CFG	0x60	t_rddata_en	[23:16]	RW	Read-data-enable latency
CTRLR_CFG	0x60	rd_in_order	[24]	RW	Force in-order reads
CTRLR_CAP	0x64	cap_lookahead_max	[3:0]	RW	Advertised OOO look-ahead depth
CTRLR_CAP	0x64	cap_synth_mask	[7:4]	RW	Advertised feature mask
DFI_TUNING	0x68	cmd_delay	[3:0]	RW	Live DFI cmd delay (reset 1) — no rebuild
DFI_TUNING	0x68	rddata_	[7:4]	RW	Live DFI read-data

Register	Offset	Field	Bits	Access	Description
		delay			delay (reset 0)

6.6 a7ddrphy calibration passthrough (hardware only)

Indirect access to the a7ddrphy's read/write-leveling knobs; firmware-driven, no HW FSM.
Meaningless in sim (the stub ignores it). Knob map:
rtl-vivado/a7ddrphy/a7ddrphy_csr_map.txt.

a7ddrphy calibration passthrough

Register	Offset	Bits	Access	Description
PHY_CSR_ADDR	0x80	[9:0]	RW	a7ddrphy CSR word index
PHY_CSR_WDATA	0x84	[31:0]	RW	Value to write
PHY_CSR_CTRL	0x88	[0] pulse	W	Drive one CSR-bus write
PHY_CSR_RDATA	0x8C	[31:0]	R	dat_r for the current PHY_CSR_ADDR

6.7 Engine config (WR 0x100–0x128, RD 0x180–0x1A8)

The read block mirrors the write block at +0x80.

Write-engine config (read engine identical at +0x80)

Register	Offset	Field	Bits	Description
WR_START_ADDR	0x100	value	[31:0]	Start address
WR_STRIDE_0/1	0x104 / 0x108	value	[23:0]	Signed strides
WR_WRAP_MASK_0 /1	0x10C / 0x110	value	[31:0]	Address wrap masks
WR_BLEN_TXN	0x114	burst_len	[7:0]	AXI burst length
WR_BLEN_TXN	0x114	txn_count	[23:8]	Transaction count
WR_BLEN_TXN	0x114	gap	[27:24]	Inter-burst gap
WR_AXI_ATTR	0x118	axi_id	[7:0]	AXI ID
WR_AXI_ATTR	0x118	id_mode	[9:8]	0 fixed / 1 counter / 2 LFSR

Register	Offset	Field	Bits	Description
WR_AXI_ATTR	0x118	axi_size	[12:10]	AXI size (AXI_SIZE_8 = 3)
WR_AXI_ATTR	0x118	axi_burst	[14:13]	0 fixed / 1 incr / 2 wrap
WR_AXI_ATTR	0x118	data_mode	[15]	Data pattern mode
WR_LFSR_SEED	0x11C	value	[31:0]	Pattern seed (must match RD for CRC)
WR_HASH_SEED0/ 1/2	0x120–0x128	value	[31:0]	Hash seeds

6.8 Perf observability (0x1C0–0x1E8)

All 32-bit; cleared by CTRL.clear_stats, frozen by CTRL.freeze_trace.

Perf observability (bin b covers $[2^b, 2^{(b+1)})$ cycles)

Register	Offset	Field	Bits	Description
OBS_RD_{PROD,BP,STARV,IDLE}	0x1C0–0x1CC	value	[31:0]	Read data-channel meter buckets
OBS_WR_{PROD,BP,STARV,IDLE}	0x1D0–0x1DC	value	[31:0]	Write meter buckets
OBS_HIST_SEL	0x1E0	bus	[0]	0 = read, 1 = write
OBS_HIST_SEL	0x1E0	metric	[1]	0 = AR→firstR / AW→B, 1 = AR→RLAST
OBS_HIST_SEL	0x1E0	bin	[5:2]	Histogram bin index (0–15)
OBS_HIST_COUNT	0x1E4	value	[31:0]	Count in the selected bin
OBS_HIST_TOTAL	0x1E8	value	[31:0]	Total txns on the selected metric

7 Simulation / Silicon Equivalence

This project follows the shared UART-characterization methodology (bin/TBClasses/harness/): **one authored-once host program runs byte-for-byte identically on the FPGA and in a cocotb simulation.** The equivalence boundary is the ASCII W/R byte stream on the UART wire; only the

bottom ByteChannel differs — SerialChannel (pyserial) on the board vs CocotbUartChannel on the DUT pins in sim.

7.1 Testbenches

Simulation testbenches

tb_top	Wraps	Drives cfg via
ddr2_char_uart_tb_top.sv	the full ddr2_char_harness (real UART bridge + harness_csr + 1→N bridge + engines + pumice)	the real UART pins (cocotb UARTMaster/Monitor)
ddr2_char_macro_tb_top.sv	the inner ddr2_char_macro	direct ports + an APB BFM

The UART tb is the equivalence vehicle: it exposes real UART pins so the sim consumes the identical byte stream the host sends to silicon. The DFI side is wired to internal phy_dfi_* nets so the framework's DFISlavePHY + MemoryModel loopback auto-binds by prefix. **There is no a7ddrphy in sim** — it reproduces the digital DFI handshake, not the analog eye.

7.2 How the same program runs in sim

test_ddr2_char_uart.py builds the DFISlavePHY + MemoryModel backend, then:

```
r = await cocotb.external(prog)() # prog is an UNMODIFIED
pumice_master routine
```

make_uart_channel(dut, clock, clks_per_bit) returns a CocotbUartChannel that drives the UART pins; the synchronous host program runs in a worker thread under cocotb.external, and the channel's read/write are cocotb.function wrappers that step the sim. The baud is lowered for sim (UART_BAUD = 6_250_000 → CLKS_PER_BIT = 16 vs 868 on board) — bit-timing only, the byte stream is unchanged. make_sim / sim-smoke / sim-simple / sim-level run the same programs the board runs.

7.3 Match the board's exact DFI config

GEAR_RATIO = log2(DFI_RATE) and **must** match the compile-time DFI_RATE or reads corrupt. The test threads TEST_DFI_RATE / TEST_GEAR_RATIO / TEST_DRAM_BEAT_BYTES into both the SV parameters={...} and the host via drv.set_dfi_phase(...). The board is built at DFI_RATE

= 4 (GEAR = 2, AXI 64 / DRAM beat 32); a rate-2 / GEAR-1 sim will **pass while the board fails** — it skips the very paths that break. Named variants (..._smoke_rate4, ..._smoke_rate4_x16, ..._pagehit_rate4_x16, ...) exist to run the board's real geometry.

7.4 What the sim can and cannot reproduce

- **CAN:** everything digital — the controller/engines, the bridge RTL, DFI-level handshakes, register logic. This is where it earns its keep.
- **CANNOT:** analog PHY behavior. Xilinx SERDES/IDELAY in a7ddrphy do not simulate, so read-eye / DQS-gate / pin-LOC problems are silicon-only.

8 Troubleshooting

8.1 Current board state

Sim timing is closed; on-board DDR2 runs but is **gated by PHY calibration**. A clean read-leveling eye is found only at **t_phy_wrlat = 0** (bit slip 0, taps 0–11, centre tap 5). The default of t_phy_wrlat = 4 used by --simple / --char breaks the write path (“no passing tap”).

Known-good board config: t_phy_wrlat = 0, t_rddata_en = 6, rd_phase = 0, rddata_delay = 8, cmd_delay = 1.

After the runtime page-policy fix, OPEN vs CLOSE page policy moved read throughput from ~12.7 MB/s to ~112 MB/s (8.8×) and write to ~44 MB/s (3.5×). For reference, LiteDRAM on the same board reaches ~600 MB/s peak (300 MT/s memtest).

8.2 Sticky errors look like a wedge

STATUS.any_error is **sticky** and is cleared by CTRL.clear_stats, **not** soft_reset. A single failed read latches rd_error; every subsequent engine wait then returns False and looks like a hardware wedge that isn't one. Clear stats between phases. Note pumice_master.wait_engine() waits on **one** engine (write-then-read is phased); a “wait for both” would hang.

8.3 Known bugs / bounds

Known issues and bounds

Symptom	Cause / workaround
Read engine wedges at ~4790 txns	rd_cmd_cam ceiling — keep board runs $\text{txn} \leq 1024$ for clean completion; the perf rate is still valid at scale
row_major wedges immediately	every burst is a row-miss — needs the wedge fix before it can be characterized

Symptom	Cause / workaround
Intermittent 1–11 beat mismatches at scale	clean at $\text{txn} \leq 1024$
col_major stride overflow	stride $16 \text{ KB} \times \text{txn} > 8192$ exceeds 128 MB → address alias; host bounds col_major to $\text{txn} \leq 8192$
synth_mask_obs / lookahead_max_obs read 0	init never set CTRLR_CAP — advertise capability first

8.4 Cannot find the board

The USB-UART re-enumerates across reboots/replugs. `--port auto` (default) probes each `/dev/ttyUSB*`, reads `BUILD_ID`, and picks the one answering `0x44445232`. If none respond it raises “no pumice DDR2 char harness responded ... is the board powered and programmed?” — confirm the bitstream is flashed and the FTDI cable is free.

8.5 a7ddrphy is not simulatable

Verilator cannot model `OSERDESE2` / `ISERDESE2` / `IDELAYE2`, so sim connects at the DFI level with `DFISlavePHY` + `MemoryModel`. Read-eye / DQS-gate / pin-LOC problems are silicon-only. Regenerating the real PHY requires **Python 3.10 only** (migen’s CSR-name tracer breaks on 3.11/3.12):

```
uv venv --python 3.10 /tmp/litex-venv310 && source /tmp/litex-venv310/bin/activate
uv pip install migen "litex==2024.12" "litedram==2024.12" pyyaml
python3 bin/elaborate_a7ddrphy.py --out
rtl-vivado/a7ddrphy/a7ddrphy_generated.v
python3 bin/elaborate_a7ddrphy.py --dump-csr-map      # CSR offsets
for firmware
```

8.6 debug_sram trace ring is disabled on this build

`DEBUG_SRAM_WORDS` was shrunk `32768` → `512` — the full ring needed ~44 K distributed-RAM cells (2.4× over the 100T budget) and blocked `place_design`. The 256-KB address *window* is unchanged (accesses > 2 KB alias into the ring); the ring is effectively unused here. Raise it on a larger device.

8.7 LiteDRAM baseline flow

`flows-litedram-uart` is wired and lint-clean but board bring-up is gated. The default `./regen.sh` builds an empty BIOS ROM (`--no-compile-software`) so the core never asserts

`init_done` — use `./regen.sh --bios` (needs `riscv-gcc`). The host variant (`harness_csr` at base 0, no pumice APB cfg) is still TODO.